# SH-CADI — Team Guide for the Final Review

**Project:** Self-Healing Cloud Storage Deduplication Using Content-Addressable Indexing (SH-CADI)

**Group B13:** Dhruv Jayesh Kansagara · Harish Kumaar S · Prithve K C · S Sri Raghav Vatsan

**Code:** github.com/dkansagara12-alt/SH-CADI

This guide covers what the project does, how every part works, what we built (and what we did *not* build), the measured results, and the viva questions we are likely to get, with answers. Read sections 1–9 to understand the system, then go through section 14 (viva questions) and section 15 (numbers cheat-sheet) together.

---

## 1. The 30-second answer: "What is your project?"

> Cloud storage saves space with **deduplication**: every file is cut into chunks, each chunk is identified by its SHA-256 hash, and a chunk that already exists is stored only once. A **hash-table index** maps each chunk hash to where the chunk lives. That index is a single point of failure: if one entry is corrupted, every file that shares that chunk becomes unreadable.
>
> Normally, recovering a corrupted index means **replaying the whole journal and rebuilding the whole index**, even if only one record is damaged. **SH-CADI** makes the index **self-healing**: a **bucketed Merkle tree** detects the corruption, **localizes** it to the exact damaged records, and the system **recovers only those records** from a journal that is partitioned the same way as the tree. It then **verifies** that the Merkle root matches the trusted root again.
>
> At 1,000,000 records with 100 corrupted, SH-CADI rebuilt **84 records instead of 1,000,000** and its recovery step was **1,030× faster** than a full rebuild (13.7 ms vs 14.1 s). Lost chunk *data* is recovered separately with **Reed-Solomon RS(4,2)** erasure coding.

Core idea in one line: **Detect → Localize → Recover → Repair → Verify, touching only the damaged part.**

---

## 2. The problem and why it matters

- **Redundancy:** the same files are uploaded again and again (copies, versions, many users). Deduplication stores each unique chunk once.
- **The index is fragile:** after deduplication, many files depend on one chunk record. A wrong storage location, chunk ID or reference count in that record breaks all of them.
- **Silent corruption:** bit rot, hardware faults and software bugs change data without raising any error.
- **Conventional recovery is all-or-nothing:** detect that *something* is wrong, then rebuild *everything*. The cost grows with index size, not with the amount of damage.

**Goal:** make recovery cost proportional to the **damage (k)**, not to the **index size (n)**.

---

## 3. Glossary (know these cold)

| Term | Meaning in our project |
|---|---|
| **Chunk** | A fixed-size piece of a file (4 KB by default; 64 B in the small demo). |
| **SHA-256** | Cryptographic hash, 256-bit output. Same bytes → same hash. Different bytes → different hash (collisions are practically impossible). |
| **Content addressing** | The chunk's hash *is* its address/name. |
| **Deduplication** | If a chunk's hash is already in the index, store only a reference (increase `ref_count`) instead of storing the bytes again. |
| **Record / entry** | One hash-table value: `chunk_id`, `storage_location`, `size`, `ref_count`, `referenced_by`. |
| **ref_count** | How many uploaded chunks point to this unique chunk. |
| **Record digest** | SHA-256 over `hash \| chunk_id \| storage_location \| size \| ref_count`. This is what the Merkle tree protects. |
| **Merkle tree** | Binary tree of hashes. Leaves hash the data; each parent = hash(left child, right child). One root hash summarizes everything. |
| **Trusted root (anchor)** | The root value we assume is safe (in a real system it would be signed or kept elsewhere). Everything else is verified against it. |
| **Bucket** | A group of records whose hashes share the same leading bits. Each bucket is one leaf of our Merkle tree. |
| **Journal** | Append-only log of every change to the index (write-ahead log), split into **one shard per bucket**. Each line has its own checksum. |
| **Recovery locality** | `records reconstructed / total records`. Lower is better: 1.0 means everything was rebuilt. |
| **Erasure coding RS(k,m)** | Split a chunk into k data fragments and add m parity fragments. Any k of the k+m fragments rebuild the chunk. RS(4,2) = 4 data + 2 parity, tolerates 2 losses. |
| **GF(256)** | Galois field with 256 elements (bytes). Reed-Solomon does its arithmetic here: addition = XOR, multiplication uses log/exp tables. |
| **Node** | A simulated storage server (a folder). Fragment *i* of every chunk lives on node *i*. |

## 4. Architecture — how a file flows through the system

```
                        NORMAL PATH (every upload)
File ──► Chunking ──► SHA-256 ──► Hash-index lookup ──► new? store chunk : ref_count += 1
                                                          │
                                                          ├──► Journal shard of that bucket  (append line,
write-ahead)
                                                          └──► Bucketed Merkle tree          (update one
leaf→root path)

                        SELF-HEALING PATH (audit, every read, every write)
Detect ──► Localize ──► Recover ──► Repair ──► Verify
```

### Upload, step by step (`ContentAddressableStorage.upload_file`)

1. `chunk_file` splits the file into fixed-size chunks and gives each a chunk ID like `FileA.txt::chunk3::a1b2c3d4`.
2. `compute_hash` gives each chunk its SHA-256 hash.
3. **Before writing**, `before_write(hash)` verifies that this record's bucket is intact and heals it if not, so corruption is never built on.
4. `hash_table.search(hash)`:
   - **Found (duplicate):** `ref_count += 1`, add to `referenced_by`. No bytes are stored.
   - **Not found (unique):** store the chunk in cloud storage, then insert a new record.
5. The listener (`SelfHealingIndex.on_upsert`) appends a journal line, then updates the Merkle path of that bucket in O(log B).
6. The file's **manifest** (its list of chunk hashes, in order) is saved so the file can be rebuilt later. `reconstruct_file` reads the chunks back in manifest order.

---

## 5. The data structures

### 5.1 Hash table (`hash_index/hashtable.py`) — the index

- **Separate chaining:** an array of buckets; each bucket is a linked list of nodes (`key, value, next`).
- **Hash function:** first 8 hex characters (32 bits) of the SHA-256 key, mod capacity. SHA-256 output is already uniformly random, so no extra hashing is needed.
- **Initial capacity 1024; resize** (double and rehash everything) when load factor > 0.75.
- Operations: `insert`, `search`, `update`, `delete`, plus `items()` to iterate.
- **Complexity:** average O(1), worst case O(n) if every key lands in one chain. Resizing costs O(n) but is amortized O(1) per insert.

### 5.2 Phase-1 Merkle tree (`merkle/merkle_tree.py`, `merkle/integrity.py`) — now the *baseline*

- Leaves = record digests **sorted by hash**. Built bottom-up; an odd node is paired with itself.
- Supports inclusion proofs (`get_proof`, `verify_proof`).
- **Problems that made us redesign it:**
  1. Sorted leaves shift on every insert, so every change forces a **full rebuild, O(n)**.
  2. Odd-node duplication is a known weakness (two different leaf lists can give the same root).
  3. Finding *which* record changed needs a **full leaf-by-leaf scan, O(n)** (`diagnose_changes`).
- We kept it unchanged as the **conventional baseline**.

### 5.3 Bucketed Merkle tree (`merkle/bucket_merkle.py`) — the core of SH-CADI

- **Fixed shape:** a perfect binary tree with **B = 2^depth leaves**. Each leaf is a **bucket**.
- A record's bucket = the **top** `depth` **bits** of its hash prefix. A record's position never moves, so an insert changes only **one leaf and its path to the root**.
- **Leaf** = SHA-256 of `"bucket:"` + the bucket's record digests in sorted order. An empty bucket has a fixed constant hash.

- **Internal node** = SHA-256 of `"node:"` + left + right. The prefixes ("domain separation") stop a leaf from being confused with an internal node.
- **Depth is picked so each bucket holds about 4 records** (`depth_for`): 10k records → 4,096 buckets · 100k → 32,768 · 1M → 262,144. The demo uses depth 6 = 64 buckets.
- **Trust model:** only `trusted_root` is trusted. The stored leaves, internal nodes and per-record digests are *caches* that can be corrupted too.
- **Fail-stop writes:** `upsert` first checks the bucket's path against the trusted root (`_verify_path`, O(log B)). If the path is inconsistent it refuses the write (`CorruptionDetected`), so a corrupted tree can never become the new "truth".
- **Complexity:** upsert O(s log s + log B) where s ≈ 4 records per bucket; bulk build O(n + B); repairing k buckets O(k log B).

### 5.4 Sharded journal (`recovery/journal.py`) — the recovery information

- **One shard per Merkle bucket** (same bucket function). Repairing one bucket reads **one small shard**, not the whole log. This co-partitioning is one of our contributions.
- Each line is JSON: sequence number, hash, operation (`put` or `del`), the record, and a **checksum** (first 64 bits of SHA-256). A damaged line fails its checksum and is skipped.
- Written **before** the Merkle tree is updated (write-ahead).
- Backends: in-memory (benchmarks) or one file per shard (demo).
- `read_shard(b)` returns the newest valid version of every record in the shard plus its full history (newest first).

---

## 6. The self-healing engine (`recovery/healer.py: SelfHealingIndex.heal`)

### Step 1 — Detect: O(n + B)

- Build a tree from the **live hash-table records** (L) and a tree from the **stored leaves** (D). Compare both roots with the **trusted root**.
- `L ≠ trusted` → **data fault** (records were changed). `D ≠ trusted` → **tree fault** (the Merkle tree itself was damaged).
- This step must read every record once. **Every system pays this cost**, including the baselines.

### Step 2 — Localize: O(k log B)

- Walk down from the root, comparing D and L node by node. **If two nodes match, skip that entire subtree.** Only mismatching branches are followed down to the leaves.
- For each flagged bucket, compare the live records with the stored per-record digests to classify each record:
  - **changed:** the record exists but its digest differs.
  - **missing:** the record was deleted from the hash table.
  - **extra:** a spurious record that should not exist.
- Special bucket cases:
  - **Leaf-only fault:** the records are fine and only the stored leaf hash is wrong. The leaf is simply recomputed.

- **Ambiguous bucket:** both the leaf and the records disagree, so the whole bucket is rebuilt from its journal shard.

### Step 3 — Recover (nothing is modified yet)

For each damaged record, try the candidate values in this order:

1. The newest valid journal line.
2. Older journal lines.
3. A journal line with `ref_count` recomputed from the upload log (the number of times the hash appears across all file manifests).

**Verify-before-install:** a candidate is accepted **only if its digest equals the digest the authenticated Merkle bucket expects.** A stale or damaged journal line can therefore never be installed. If no candidate matches, the record is reported as **unrecoverable**, never silently "fixed".

### Step 4 — Repair: O(k log B)

Write the accepted records into the hash table, delete spurious ones, and recompute only the flagged leaves and their ancestors.

### Step 5 — Verify: O(k log B)

For every flagged bucket: the live digests must equal the stored digests, and the leaf must chain up to the trusted root. The repaired root must equal the trusted root. Optionally (`verify_full=True`), a complete re-audit is run.

### Automatic triggers

- `verified_get(hash)`: every **read** checks its bucket in O(s + log B) and heals just that record if it is damaged (about 33–55 µs measured).
- `before_write(hash)`: every **write** does the same check first, so a corrupted `ref_count` is never incremented and written back as truth.

### Worked example (from section 3 of the demo, 25 records, 64 buckets)

```
before   : H → FileB.txt::chunk11
injected : H → Chunk_X                       (fault injection)
Detect   : live root ≠ trusted root
Localize : bucket 53 of 64; 13 tree nodes compared; 3 of 25 records examined
Recover  : 1 journal shard read; candidate digest matches the tree
Repair   : 1 record rebuilt; its Merkle path updated
Verify   : repaired root = trusted root; full audit intact
after    : H → FileB.txt::chunk11            locality = 1/25 = 0.04
```

> **Important principle (from the master prompt):** a Merkle hash **never reconstructs data**. It only tells us *whether* something is right and *where* it is wrong. The correct value comes from the journal (metadata) or from erasure-coded fragments (chunk data). The Merkle tree only **authenticates** the rebuilt value.

---

## 7. Chunk-data recovery (`recovery/erasure.py`, `storage/protected_store.py`, `recovery/chunk_healer.py`)

Metadata corruption and lost chunk bytes are **separate problems with separate mechanisms**:

| | Metadata (index records) | Chunk data (the bytes) |
|---|---|---|
| What authenticates it | Merkle tree + trusted root | The chunk's own SHA-256 (its address) |
| Where the correct value comes from | Journal shard (+ upload log) | Surviving erasure-coded fragments |
| Healer | `SelfHealingIndex.heal` | `ChunkHealer.heal` |

### Reed-Solomon RS(4,2)

- A chunk is padded and split into **4 data fragments (C1–C4)**. **2 parity fragments (P1, P2)** are computed with a **Cauchy matrix over GF(256)**.
- **Systematic:** the first 4 fragments are the original bytes, so with no damage, decoding is just concatenation.
- **MDS property:** *any* 4 of the 6 fragments rebuild the chunk. Decoding inverts the 4×4 sub-matrix for the surviving fragments (Gauss-Jordan elimination in GF(256)) and multiplies.
- **Storage overhead 1.5×**, compared with 3× for 3-way replication with the same 2-loss tolerance. The price is **repair amplification**: rebuilding one fragment reads 4 fragments.

### Storage layout and checks

- Fragment *i* of every chunk lives on **node i** (`node_0` … `node_5`), so one node failure costs at most one fragment per chunk.
- Each fragment file = **8-byte checksum | 4-byte chunk size | payload**. The checksum only classifies a fragment as ok, corrupt or missing.
- **Final authority:** `SHA-256(rebuilt chunk) == chunk hash`. `decode_verified` tries other 4-fragment subsets if a damaged fragment slipped past its checksum (silent corruption).
- **Degraded read:** a read still succeeds while 4 or more fragments are healthy, and it triggers a repair automatically.
- **ChunkHealer:** *scrub* (classify every fragment, check the content hash) → *recover* (decode from healthy fragments) → *repair* (**rewrite only the bad fragments**) → *verify*.
- **3 or more fragments lost:** reported as **unrecoverable**, never faked.
- `heal_all()` heals the **metadata first**, because the index decides which chunks must exist, and then the chunks.

---

## 8. Fault injection (`fault_injection/injector.py`)

Seeded (reproducible) and logged with ground truth, so we can check that localization found *exactly* the injected faults.

| Category | Faults |
|---|---|
| Metadata | change storage location · change chunk reference (`H → Chunk_X`) · change `ref_count` · delete entry · insert spurious entry |

| Category | Faults |
|---|---|
| Merkle tree | corrupt a stored leaf |
| Chunk data | delete a chunk · delete or corrupt a fragment (optionally *silent*, with a valid checksum) · fail a whole node |
| Recovery information | damage a journal line · wipe a journal shard |
| Multiple | `inject_random(k)` on k distinct records, plus optional leaf faults |

---

# 9. Evaluation

## 9.1 The three systems (all use the same hash table and the same journal)

1. **Conventional (full rebuild)** — the baseline the master prompt defines. Detect = rebuild the Phase-1 tree and compare roots. Localize = leaf-by-leaf full scan. Recover = **replay the entire journal** into a new table. Verify = rebuild the tree again.
2. **Conventional + targeted repair** — a *stronger* baseline we added on purpose so the comparison is fair. It repairs only the damaged records, but still needs O(n) scans to find them and to verify.
3. **SH-CADI** — bucketed tree + co-partitioned journal + localized heal.

## 9.2 Setup

- Index sizes 10,000 / 100,000 / 1,000,000 records; k = 1, 10, 100 faults; corruption rates 0.1 %–25 %.
- Each (system, size) runs in a **fresh process** for clean memory measurement. Repetitions: 5 at 10k, 3 at 100k, **1 at 1M**.
- **Machine:** Windows 11 PC, Python 3.13, 24 GB RAM; the code is single-threaded. The full run takes about 4 minutes.
- Synthetic records, in-memory journal, no disk or network I/O modelled.
- All numbers are in `benchmark_results/RESULTS.md` and `results.json`. **None are estimated.**

## 9.3 Key results (1,000,000 records, k = 100)

| Metric | Conventional | Conv. + targeted | SH-CADI |
|---|---:|---:|---:|
| Lookup hit (µs) | 0.59 | 0.58 | 0.98 |
| Insert (µs) | 8.6 | 7.5 | **58.0** (cost) |
| Memory added (MB) | 1,336 | 1,336 | 1,281 |
| Clean full verification (s) | 2.76 | 2.73 | 3.44 |
| Detection (s) | 2.70 | 2.66 | 2.96 |
| **Localization (ms)** | 2,779 | 2,685 | **7.3** |
| **Recovery phase (ms)** | 14,107 | 5,389 | **13.7** |
| Records reconstructed | 1,000,000 | 84 | 84 |
| Recovery locality | 1.0 | 8.4e-5 | 8.4e-5 |
| Success rate | 100 % | 100 % | 100 % |

| Metric | Conventional | Conv. + targeted | SH-CADI |
|---|---|---|---|
| Upload throughput (chunks/s) | 67,011 | 67,011 | **21,590** (cost) |
| Journal storage (MB) | 260.8 | 260.8 | 260.8 |

**Headline numbers:**

- Recovery phase **1,030× faster** than full rebuild and **393× faster** than targeted repair.
- End to end, including detection: **5.7×** faster than full rebuild (2.97 s vs 16.8 s) and **2.7×** faster than targeted repair.
- **Why only 84 reconstructed out of 100 faults?** 16 of the injected faults were *spurious entries*; they are **removed**, not reconstructed.

### 9.4 Other findings

- **Scalability:** detection grows linearly for all three systems. SH-CADI's recovery phase only went from 4.7 ms to 13.7 ms over 100× more data (k = 100).
- **Corruption-rate sweep (100k records):** SH-CADI is 72× faster than targeted repair at 0.1 % corruption, but only **1.35× faster at 25 %**. On an earlier 1-CPU Linux run the two were tied at 25 %. **The advantage shrinks as damage grows.**
- **Journal shard wiped + 10 faults:** full rebuild **lost 2–6 healthy records**, because it trusts the journal for everything. Targeted repair and SH-CADI lost **none**. This robustness comes from *localized recovery*, not from the Merkle tree itself.
- **Read-path healing:** a read that finds and fixes a corrupted record takes 33 / 48 / 55 µs at 10k / 100k / 1M records.
- **Localization accuracy:** 100 % of trials found exactly the injected faults.

### 9.5 Costs (be upfront about these)

- Inserts are **6.7× slower** (58 vs 8.6 µs) because every write verifies and updates a Merkle path.
- Uploads are **3.1× slower** (21,590 vs 67,011 chunks/s). A plain hash table with no journal reaches 132,729.
- At 1M records on this PC, plain lookups (0.98 vs 0.59 µs) and a clean full verification (3.44 vs 2.76 s) were also slower.
- Same journal size and **no extra memory** (1,281 vs 1,336 MB).

---

## 10. Existing techniques vs our contribution (be honest about this)

**Existing — we use them, we do not claim them:** SHA-256 content addressing (Venti, Data Domain), hash tables, Merkle trees, top-down Merkle descent (Dynamo / Cassandra anti-entropy), write-ahead logs, Reed-Solomon coding, replication.

**What SH-CADI contributes (as a design composition, not new primitives):**

1. **Co-partitioned detection and recovery:** the Merkle bucket partition is also the journal-shard partition, so one damaged bucket means one small journal read.

2. **Verify-before-install repair:** a rebuilt record is accepted only if its digest matches the authenticated tree. Several sources are tried, and failures are reported.
3. **Read and write guards** that heal a record before it is used or overwritten.
4. **Recovery locality** as the headline evaluation metric, plus a measured comparison against two baselines.

**Novelty caveat:** our literature check was limited (8 searches). It does **not** prove no one has done this. Say "we did not find" rather than "we are the first".

---

## 11. Limitations (say these before the examiner does)

- Single-threaded pure Python. Absolute times would be far lower in C/Rust, so **ratios** matter more than absolute times.
- The 1M row is a single run, with no variance estimate.
- The journal is in memory: disk and network I/O are not modelled.
- Faults are synthetic and uniformly random; correlated damage is tested only through the wiped-shard scenario.
- Threat model: **accidental** faults with a trusted root. No protection against a malicious attacker who can also change the root.
- Detection is still O(n).
- `referenced_by` is not covered by the digest and is restored best-effort.
- No concurrency control (single writer).
- Fixed-size chunking (no content-defined chunking).

## 12. Changes since the mid-review

- The Phase-1 sorted-leaf Merkle tree was replaced by the **bucketed fixed-shape tree**. The old tree is now the baseline.
- Built since the mid-review: localization, the journal, the healer, verified reads and writes, Reed-Solomon chunk recovery, the fault injector, the benchmark harness, one combined demo, and 66 unit tests.
- Mid-review novelty #2, **fault-tolerant chunk placement**, became RS(4,2) with one fragment per node.
- Mid-review novelty #3, **dynamic redundancy optimization**, was **not built**. It is listed as future work, because we chose to keep the focus on the self-healing index.

## 13. How to run (for the live demo)

```
git clone https://github.com/dkansagara12-alt/SH-CADI.git
cd SH-CADI
pip install -r requirements.txt
python -m sh_cadi.main               # full demo, ~1–2 min, ends with a 15/15 PASS scorecard
python -m sh_cadi.main --quick       # same without the 100k-record scale section (~5 s)
python -m unittest discover tests    # 66 tests
python -m sh_cadi.benchmark.run_all --out benchmark_results         # full evaluation (~4 min here)
python -m sh_cadi.benchmark.report benchmark_results/results.json   # regenerate RESULTS.md + plots
```

---

# 14. Possible viva questions — with answers

## A. Basics and motivation

1. **What is deduplication?** Storing each unique chunk once. Later copies are replaced by a reference, and `ref_count` is increased.
2. **Why chunk files instead of deduplicating whole files?** Two files that share only part of their content (for example two versions of a document) can still share chunks.
3. **Why SHA-256?** It is collision-resistant, so equal hashes mean equal content with overwhelming probability. The 256-bit output also gives uniformly distributed hash-table and bucket indices for free.
4. **What if two different chunks have the same SHA-256 (a collision)?** The birthday bound puts the chance at around $2^{-128}$ for realistic data sizes. Real systems (Venti, Git, ZFS dedup) rely on the same assumption. Our system would treat the second chunk as a duplicate.
5. **Fixed-size or content-defined chunking?** Fixed-size. It is simple, but inserting one byte shifts every later chunk boundary. Content-defined chunking (Rabin fingerprints) avoids that and is future work.
6. **Where is the "cloud"?** It is simulated with local directories. The protected store uses one directory per node (`node_0` … `node_5`).
7. **What exactly is the problem you solve?** Recovering a corrupted deduplication *index* without rebuilding the entire index.

## B. Data structures and complexity

8. **Which data structures did you implement?** A chained hash table, the Phase-1 Merkle tree (now the baseline), a bucketed fixed-shape Merkle tree, a sharded checksummed journal, and Reed-Solomon over GF(256).
9. **Hash table complexity?** Average O(1) for insert, search, update and delete; worst case O(n) when all keys collide. Resizing at load factor 0.75 doubles capacity and is amortized O(1).
10. **Why separate chaining, not open addressing?** Deletion is simple (no tombstones), and performance degrades gracefully at high load.
11. **How is the hash-table index computed?** The first 8 hex characters (32 bits) of the SHA-256 key, mod capacity. The key is already a uniform hash.
12. **What is a Merkle tree and why use one?** A tree of hashes whose single root summarizes all the data. Any change to any leaf changes the root, and comparing nodes top-down tells you *where* the change is.
13. **Why did you redesign the Phase-1 Merkle tree?** Its sorted leaves shift on every insert, so every update needs an O(n) rebuild. Its odd-node duplication is a known weakness. And localizing a change needed an O(n) scan. The bucketed tree has a fixed shape, so an update is O(log B) and localization is O(k log B).
14. **How is a record's bucket decided?** The top `depth` bits of the hash prefix. The bucket is fixed by the hash, so it never moves.
15. **How many buckets, and why about 4 records per bucket?** `depth_for` picks the smallest depth so that buckets hold at most about 4 records on average. Small buckets keep the in-bucket work small ($s \approx 4$) while the tree stays a reasonable size ($B \approx n/4$). Bigger buckets mean more work per damaged bucket; smaller ones mean a deeper tree and more memory.

16. **What is domain separation?** Leaves are hashed with the prefix `"bucket:"` and internal nodes with `"node:"`, so a leaf can never be confused with an internal node.
17. **Complexity of each healing step?** Detect O(n + B); Localize O(k log B + flagged bucket sizes); Recover O(journal lines of the k shards); Repair O(k log B); Verify O(k log B). Here n = records, B = buckets, k = damaged buckets.
18. **Why is detection still O(n)? Couldn't you avoid it?** To know that *no* record changed, every record must be read and hashed at least once, because corruption bypasses the API. Every system pays this. We reduce it in practice with per-access checks (`verified_get`, `before_write`) at O(s + log B). Incremental or sampled scrubbing is future work.
19. **Space overhead of the tree?** O(n + B): one digest per record plus about 2B tree nodes. We measured no extra memory over the conventional system (1,281 vs 1,336 MB at 1M records).

## C. The healing algorithm

20. **Walk us through Detect → Localize → Recover → Repair → Verify.** See section 6. In short: compare roots; descend only mismatching branches; fetch candidates from that bucket's journal shard; accept only digest matches; write them back; recompute the path; check that the root equals the trusted root.
21. **How does the system know what the correct record is?** The Merkle tree stores the expected *digest* of every record, and that digest is authenticated through the bucket leaf up to the trusted root. The full record *value* comes from the journal. A journal candidate is installed only if its digest equals the expected digest.
22. **Can the Merkle tree rebuild the data by itself?** No. A hash cannot be inverted. The tree only detects, localizes and verifies. The value comes from the journal, or from erasure-coded fragments for chunk data.
23. **What if the journal line is also corrupted?** Its checksum fails and the line is skipped. Then older journal lines are tried, then an older line with `ref_count` recomputed from the upload log. If nothing matches, the record is reported as **unrecoverable**, not guessed. The demo's section 4 shows this.
24. **What if the Merkle tree itself is damaged?** A **leaf-only** fault (records fine, stored leaf wrong) is fixed by recomputing the leaf. If both the leaf and the records disagree, the bucket is **ambiguous** and is rebuilt from its journal shard. Either way the final root is checked against the trusted root.
25. **What if the trusted root is corrupted?** It is our trust anchor, by assumption. It is only 32 bytes, so a real system would replicate or sign it and keep it separately. Protecting it is outside our threat model, and we say so.
26. **What is a spurious entry and how is it handled?** A record that should not exist (a "ghost" hash). Localization classifies it as *extra*, and repair deletes it. That is why 100 faults gave 84 reconstructions plus 16 removals.
27. **What are verified reads and writes?** `verified_get` checks the record's bucket path before returning a value, and heals that record if needed. `before_write` does the same before an upload changes a record. This stops corrupted values from being written back as truth.
28. **Why is the journal sharded by bucket?** So that one damaged bucket needs only one small shard read. A single global log would have to be replayed or scanned in full.
29. **Write-ahead — what does it mean here?** The journal line is appended before the Merkle tree is updated, so the recovery information always exists for every authenticated state.

30. **What does "recovery locality" mean, and why does targeted repair have the same locality as SH-CADI?** Locality counts the records *reconstructed*. Targeted repair also rebuilds only the damaged records. The difference is the *cost of finding and verifying* them: targeted repair needs O(n) scans (2.7 s at 1M), while SH-CADI's tree descent takes 7.3 ms.
31. **What does the Merkle tree buy you over targeted repair, then?** Localization and verification without extra O(n) passes, plus O(log B) authenticated reads and writes.

## D. Chunk recovery and erasure coding

32. **Metadata corruption vs chunk loss: what's the difference?** Metadata corruption means the index points to the wrong place or has wrong counts; it is repaired from the journal and verified by the Merkle root. Chunk loss means the bytes are gone or damaged; they are rebuilt from fragments and verified by the chunk's own SHA-256.
33. **How does Reed-Solomon work, in simple terms?** The data is split into k pieces and m extra pieces are computed as linear combinations in GF(256). Since any k pieces give k independent equations, you can solve for the original data.
34. **Why GF(256)?** Every byte is a field element, so the arithmetic stays within bytes. Addition is XOR and multiplication uses log/exp tables. Every non-zero element has an inverse, which is what makes the matrix inversion possible.
35. **Why a Cauchy matrix?** Every square sub-matrix of a Cauchy matrix is invertible, which guarantees that *any* k fragments can rebuild the chunk (the MDS property).
36. **RS(4,2) vs 3-way replication?** Both survive 2 losses. RS uses 1.5× storage, replication 3×. But an RS repair reads 4 fragments to rebuild one, while replication just copies.
37. **What if 3 of the 6 fragments are lost?** That is beyond the tolerance (m = 2). The chunk is reported as unrecoverable. The demo shows this.
38. **What is silent corruption and how do you catch it?** A fragment whose bytes are wrong but whose checksum still looks valid. `decode_verified` checks SHA-256 of the result and tries other 4-fragment subsets until the content hash matches.
39. **Why heal metadata before chunks?** The index decides which chunks must exist. A deleted record would hide its chunk from the chunk scrub.

## E. Evaluation and results

40. **What is your baseline, and is the comparison fair?** Conventional full rebuild is the baseline the project brief defines. We added a stronger targeted-repair baseline because full rebuild was easy to beat. All three use the same hash table and the same journal, so only the recovery method differs.
41. **Your recovery is 1,030× faster but end to end only 5.7×. Why?** Detection takes about 3 s at 1M records and is O(n) in every system. SH-CADI speeds up everything *after* detection.
42. **Where does SH-CADI lose?** Inserts are 6.7× slower and uploads 3.1× slower. At 1M records on our PC, plain lookups and a clean full verification were also slower. Near 25 % corruption the advantage almost disappears.
43. **Why does the advantage shrink at high corruption rates?** When most buckets are damaged, the descent visits most of the tree and recovery approaches a rebuild anyway.
44. **Is 6.7× slower inserts acceptable?** It depends on the read/write mix and how often corruption happens. Our goal was "acceptable normal performance". Uploads still reach about 21,600 chunks/s

(about 88 MB/s) in pure Python. We report the cost openly.

45. **How did you check that localization is correct?** The injector logs ground truth, and every trial compares the localized set with the injected set: 100 % exact.
46. **How many repetitions? Are the results reliable?** 5 runs at 10k, 3 at 100k, 1 at 1M. The 1M row has no variance estimate, which we state as a limitation. The ratios are stable across two machines (Linux and Windows).
47. **Why did the shard-loss test make full rebuild fail?** Full rebuild replays the journal for every record, so records whose shard was wiped disappear. Localized methods never touch healthy buckets.
48. **What is the memory overhead?** None measured: SH-CADI used 1,281 MB vs 1,336 MB at 1M records. The journal is 260.8 MB in all three systems.

## F. Implementation

49. **Why Python?** It is fast to build and easy to read, and the master prompt specified it. Absolute times would be lower in C/Rust, which is why we compare ratios.
50. **How did you test it?** 66 unit tests covering the hash table, Merkle proofs, the bucket tree, detection and localization per fault type, healing, unrecoverable cases, stale journal lines, Reed-Solomon over every k-subset, node failure, silent corruption, the integrated system, and the benchmark harness. The demo adds a 15-check scorecard.
51. **Does it handle concurrent writers?** No. It is single-threaded with a single writer. Concurrency control is future work.
52. **How is the benchmark isolated?** Each (system, size) pair runs in its own process with garbage collection disabled during timing, and seeded faults.

## G. Tough / critical questions

53. **Isn't this just ZFS?** ZFS also uses a checksum tree and repairs from redundancy, so the *idea* is related. ZFS works on filesystem blocks. We apply it to a *deduplication index*, with a bucket-partitioned journal and verify-before-install repair. We do not claim to be the first self-healing storage system.
54. **Isn't this just Dynamo's Merkle anti-entropy?** Top-down descent is borrowed from Dynamo and Cassandra. They compare two replicas; we compare the live index with a trusted root and then *recover* from a co-partitioned journal.
55. **Why not use a database with a WAL?** A database WAL recovers from crashes by replaying. It does not detect silent corruption in already-committed index records, or localize it.
56. **What happened to dynamic redundancy optimization from your mid-review?** We dropped it to keep the focus on the self-healing index, which is the core contribution. It is future work.
57. **What would you do with more time?** Sub-linear detection (incremental or sampled scrubbing), a disk-backed journal, a compiled implementation, correlated and adversarial faults, concurrency, content-defined chunking, dynamic redundancy, and a systematic literature survey.

---

# 15. Numbers cheat-sheet (memorize)

| Fact | Value |
|---|---|
| Recovery phase at 1M, k = 100 | **13.7 ms** vs 14.1 s (full rebuild) vs 5.39 s (targeted) |
| Speed-up, recovery phase | **1,030×** vs full rebuild · **393×** vs targeted |
| Speed-up, end to end | **5.7×** vs full rebuild · **2.7×** vs targeted |
| Records reconstructed | **84** vs 1,000,000 (16 spurious removed) |
| Recovery locality | $8.4 \times 10^{-5}$ vs 1.0 |
| Localization at 1M | 7.3 ms vs 2.78 s |
| Detection at 1M | ≈ 3 s, O(n) for every system |
| Insert cost | 58 µs vs 8.6 µs (6.7× slower) |
| Upload throughput | 21,590 vs 67,011 chunks/s (3.1× slower) |
| Memory at 1M | 1,281 MB vs 1,336 MB |
| Journal at 1M | 260.8 MB (same for all) |
| Read-path heal | 33 / 48 / 55 µs at 10k / 100k / 1M |
| 25 % corruption | 944 ms vs 1,280 ms (only 1.35×) |
| Buckets | 4,096 / 32,768 / 262,144 for 10k / 100k / 1M |
| RS(4,2) | 4 data + 2 parity · tolerates 2 · 1.5× storage |
| Tests / demo | 66 unit tests · 15/15 demo checks |